SALPHOENIX®1A 系列 FPGA 配置测试验证

1 术语/缩略词

ALFPGA®（以下简称 FPGA） ：Field Programmable Gate Array 现场可编程门阵列

TangDynasty®（以下简称 TD）：Tang Dynasty 安路 FPGA 编程软件

JTAG ：Joint Test Action Group 联合测试工作组

2 关于本手册

PH1A400_CONFIG_FUN_BOARD 可覆盖 PH1A400 各种配置模式。本手册将介绍 PH1A400 配置测试平台及其各测试模式验证。

3 验证平台

硬件：ARM+DDR3，CPU 主频 667MHz；

内核：Linux3.17；

启动方式：SD；

数据传输：Ethernet；

调试接口：JTAG & UART。

验证平台支持如下配置模式：

Linux 命令行	配置模式	实现
CPU GPIO 模拟 JTAG 配置		
./Linux_PH1A400_CFG.elf 1 Test.aje	JTAG	CPU aje 模拟
./Linux_PH1A400_CFG.elf 2 Test.tde	JTAG	CPU tde 模拟
CPU GPIO 模拟从动配置		
./Linux_PH1A400_CFG.elf 3 PH4Led_Compress.bin	Slave SerialX1	CPU GPIO 模拟
./Linux_PH1A400_CFG.elf 4 PH4Led_Compress.bin	Slave PCMX8	CPU GPIO 模拟
./Linux_PH1A400_CFG.elf 5 PH4Led_Compress.bin	Slave PCMX16	CPU GPIO 模拟
./Linux_PH1A400_CFG.elf 6 PH4Led_Compress.bin	Slave PCMX32	CPU GPIO 模拟
./Linux_PH1A400_CFG.elf 7 bit0525_pcmx8.vec bit0525_pcmx8_ver.vec	Slave PCMX8 ReadBack	CPU GPIO 模拟
./Linux_PH1A400_CFG.elf 8 bit0525_pcmx16.vec bit0525_pcmx16_ver.vec	Slave PCMX16 ReadBack	CPU GPIO 模拟
./Linux_PH1A400_CFG.elf 9 PH4LED_spx32_prog.vec PH4LED_spx32_verify.vec	Slave PCMX32 ReadBack	CPU GPIO 模拟
硬件模拟主动配置		
./Linux_PH1A400_CFG.elf 10 PH4Led_Compress22M_Full.bin	Master PCM X8	FPGA 模拟

Preliminary Technical Report

./Linux_PH1A400_CFG.elf 11 PH4Led_Compress22M.bin	Master PCM X16	FPGA 模拟
./Linux_PH1A400_CFG.elf 12 PH4Led_Compress22M_Full.bin	Master Serial X1	FPGA 模拟
硬件模拟从动配置(100MHz)		
./Linux_PH1A400_CFG.elf 13 PH4Led_Compress.bin	Slave SerialX1	FPGA 模拟
./Linux_PH1A400_CFG.elf 14 PH4Led_Compress.bin	Slave PCMX8	FPGA 模拟
./Linux_PH1A400_CFG.elf 15 PH4Led_Compress.bin	Slave PCMX16	FPGA 模拟
./Linux_PH1A400_CFG.elf 16 PH4Led_Compress.bin	Slave PCMX32	FPGA 模拟
从动串行级联		
./Linux_PH1A400_CFG.elf 21 led_bypass0618_1.bin led_bypass0618_2.bin	Slave Serial Cascade	CPU GPIO 模拟

注 1：由于平台支持回读，*.vec 文件超过 100MB(非压缩)，因此需用 sd 卡进行启动。

注 2：系统加载，可通过命令行对各种配置进行测试验证 TDE/AJE/SSx1/PCMx8/PCMx16/PCMx32/MPx8/MPx16/MSx1 及相关回读。

注 3：JTAG(AJE/TDE)测试采用 PH1A100 配置平台测试测试 PH1A400。

Preliminary Technical Report

4 配置模式及其验证测试

硬件：ARM+DDR3，CPU 主频 667MHz；

4.1 平台实现

面向用户(移植)：通过软件模拟，CPU 读取内存数据，将配置数据通过 GPIO 下发出去，如图 4-1 所示。

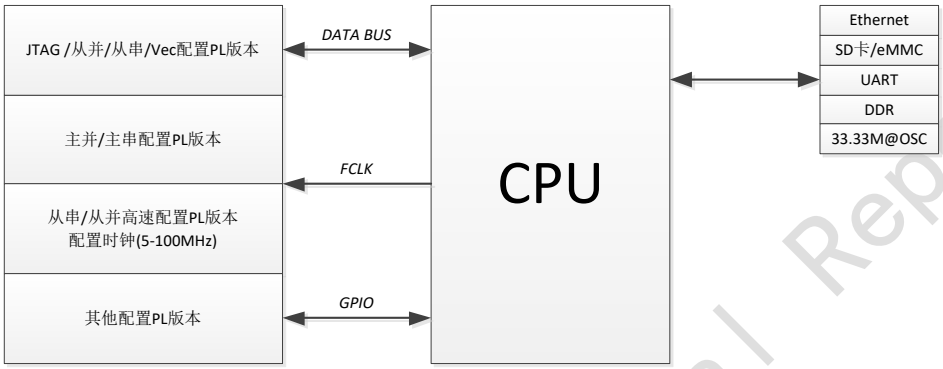

图 4-1 CPU GPIO 模拟配置实现框图

面向测试(性能)：以逻辑为 Master，通过 CPU 动态配置时钟、相位，然后通过 DMA，直接读取 DDR 上的版本数据，进行配置，如图 4-2 所示；

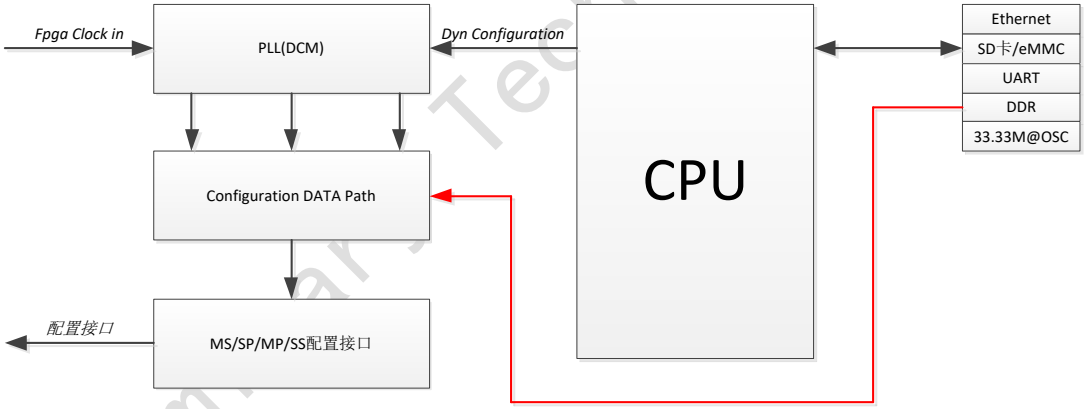

图 4-2 硬件 FPGA 模拟配置实现框图

4.2 MSPI 配置及其测试

4.2.1 MSPI 配置

MSPI 硬件连接如图 4-3 所示。

Preliminary Technical Report

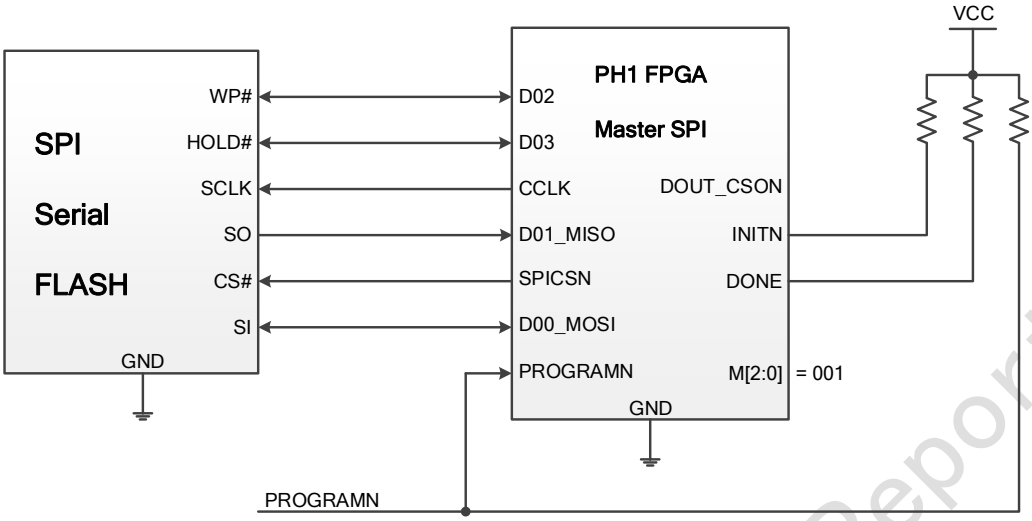

图 4-3 PH1A400 系列 FPGA SPI x4 配置

在 TD 软件中，可通过 Process->Properties->Generate Bitstream 设置 mclk_freq 及其 boot_mode，如图 4-4、4-5 所示。

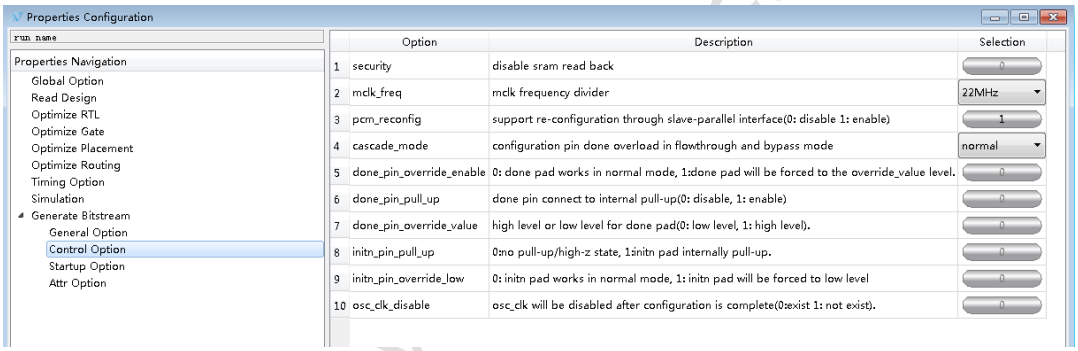

图 4-4 mclk_freq 配置

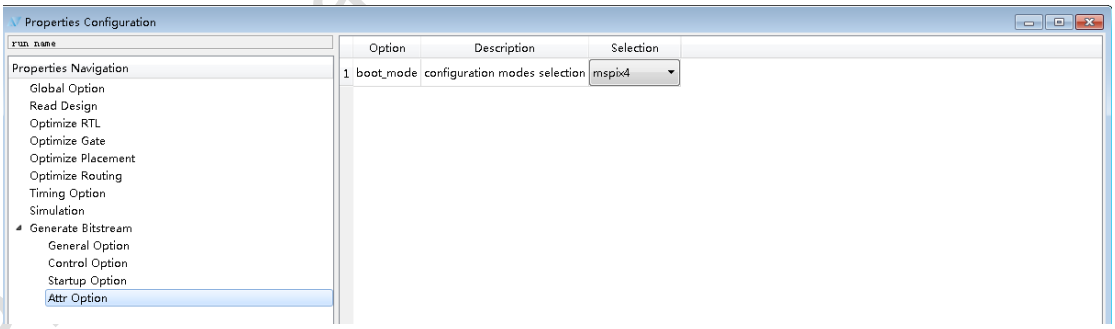

图 4-5 boot_mode 配置

4.2.2 测试验证

TD 产生 Bit 文件，可通过 download 烧写到 flash 中，测试罗列如下表所示。

测试项	测试结果

Freq.=2MHz	实测 1.86MHz（OSC 存在误差）
Freq.=3MHz	实测 2.71MHz（OSC 存在误差）
Freq.=5MHz	实测 4.58MHz（OSC 存在误差）
Freq.=11MHz	实测 9.96MHz（OSC 存在误差）
Freq.=22MHz	实测 19.90MHz（OSC 存在误差）
Freq.=33MHz	实测 29.86MHz（OSC 存在误差）
Freq.=44MHz	实测 39.68MHz（OSC 存在误差）
x4 44MHz 加载时长(非压缩模式，最短时长)	604.6ms(OSC 存在误差)
x1 2MHz 加载时长(非压缩模式，最长时长)	51.5s(OSC 存在误差)

PROGRAMN 复位后->INITN 拉高之间时长，如图 4-6 所示

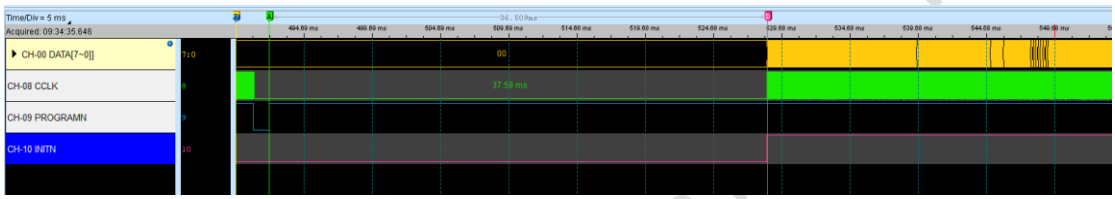

图 4-6 复位初始化时长

4.3 从动串行配置及其测试

4.3.1 从动串行配置

Slave Serial 配置模式如图 4-7 所示，CCLK 由外部上位机（如 CPU、MCU）提供。

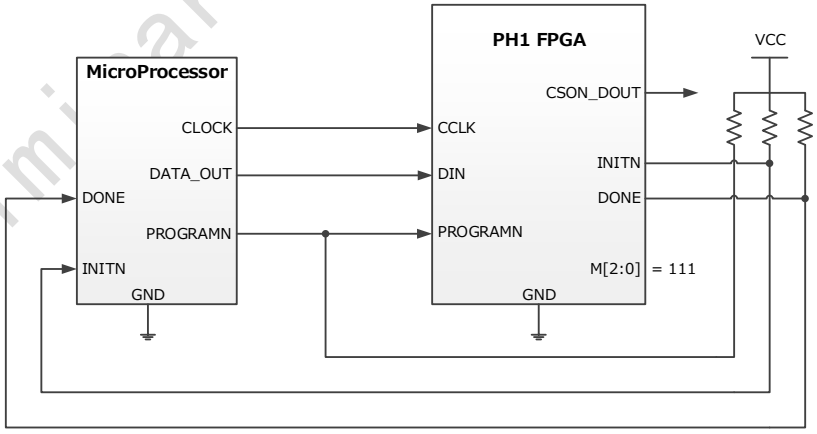

图 4-7 PH1A400 系列 FPGA Slave Serial 配置模式

对于 Slave Serial 配置，M[2:0]=111；

CCLK，由外部处理器提供；

DIN，串行配置数据；

Preliminary Technical Report

PROGRAMN，逻辑复位；

INITN，初始化完成指示信号，拉高后，器件可进行配置。

4.3.2 从动串行级联配置

从动串行级联，在硬件上将第一级 DOUT，接至第二级 DIN，软件上做如图 4-8 配置。

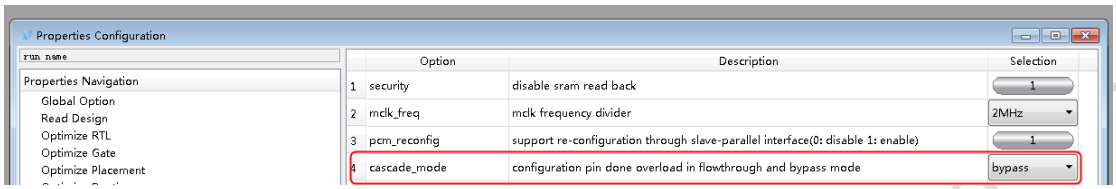

图 4-8 TD 配置

PH1A400 配置板支持级联测试，在 2 级级联时，需要将拨码开关做如图 4-9 所示设置。

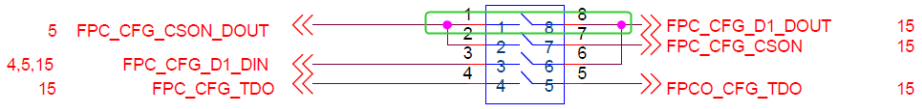

图 4-9 拨码开关配置

对于从动串行级联，可通过 tcl 命令生成 bin 文件。软件运行到 generate bitstream 阶段，将其设计成 Bypass 模式，和多片同步启动模式。

第一级

```
bitgen -bit xxx2.bit -version 0X00 -bin led_bypass0618_1.bin -g { cascade_mode:01 sync_ext_done:1 }
```

第二级

```
bitgen -bit xxx2.bit -version 0X00 -bin led_bypass0618_2.bin -g { cascade_mode:00 sync_ext_done:0 }
```

4.3.3 测试验证

```
./Linux_PH1A400_CFG.elf 3 PH4Led_Compress.bin
```

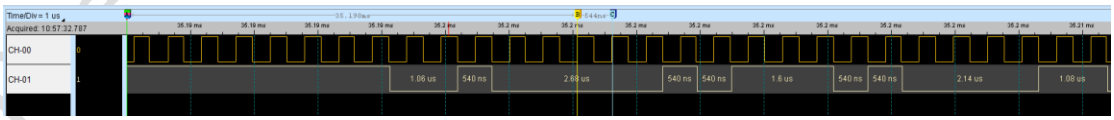

图 4-10 串行加载波形

```
./Linux_PH1A400_CFG.elf 21 led_bypass0618_1.bin led_bypass0618_2.bin
```

Preliminary Technical Report

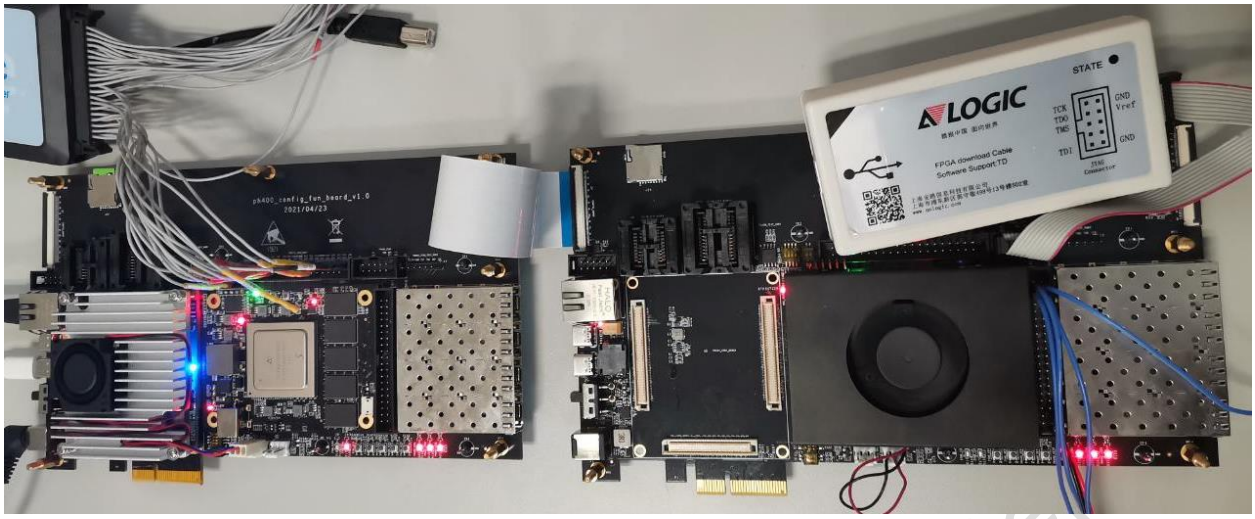

图 4-11 串行级联

4.4 从动并行配置及其测试

4.4.1 从动并行配置

从动并行配置模式，CCLK 由外部控制器提供，配置方式如图 4-12 所示。

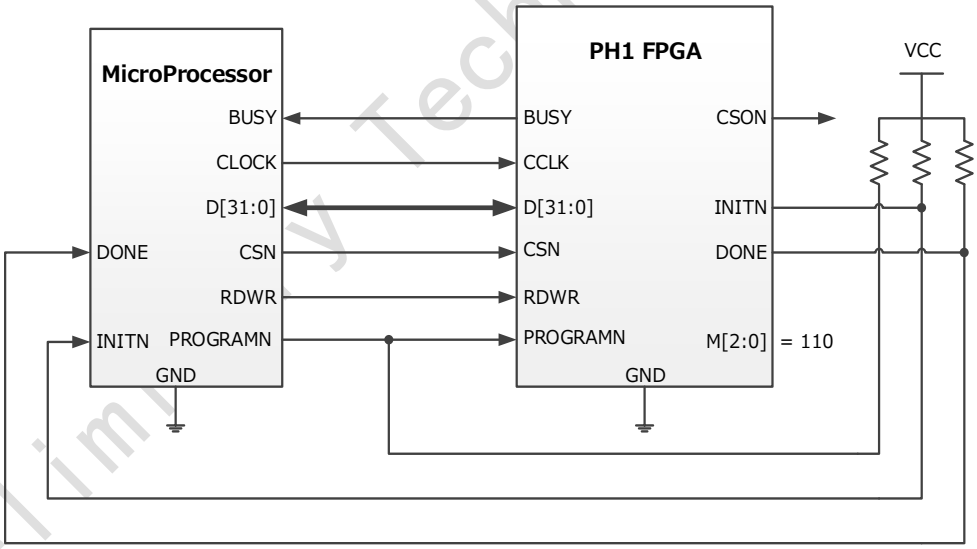

图 4-12 PH1A400 系列 FPGA 从动并行模式

对于从动并行配置，M[2:0]=110；

CCLK，由外部处理器提供；

PROGRAMN，逻辑复位；

INITN，初始化完成指示信号，拉高后，器件可进行配置；

D[31:0]，并行配置数据，根据需求可将其以 x8/x16/x32 模式送至目标器件，发送的的顺序，用下述二进制文件例举说明（此文件非码流文件，仅做说明），如图 4-13 所示。

Preliminary Technical Re

```
00000000h: FF FF FF FF DD BB 66 22 FF FF FF 02 FF FF FF FF ;
00000010h: FF FF FF FF 00 00 00 AA 11 22 00 44 FF FF FF FF ;
00000020h: FF FF FF FF FF FF FF FF CC 55 AA 33 80 00 00 00 ;
00000030h: 80 00 00 00 90 E0 00 02 40 64 30 D1 E0 5D 0B 1A ;
00000040h: 80 00 00 00 90 B0 00 02 82 02 A0 00 97 22 CF 49 ;
00000050h: 80 00 00 00 80 00 00 00 80 00 00 00 80 00 00 00 ;
00000060h: 80 00 00 00 90 C0 00 02 40 10 00 F8 F5 B7 D9 A3 ;
00000070h: 80 00 00 00 90 D0 00 02 00 00 00 00 70 EC 66 FB ;
00000080h: 80 00 00 00 90 30 00 02 00 00 00 04 12 94 F5 84 ;
00000090h: 80 00 00 00 91 70 00 02 00 00 00 00 CE B9 A6 7B ;
```

图 4-13 BIN 文件

x8 模式，按照正常的字节读出来，发出去即可；

x16 模式，其中高 8 位为 0xDD，低 8 位为 0xBB，以此类推；

x32 模式，其中第 31~24 位为 0xDD，第 23~16 位为 0xBB，第 15~8 位为 0x66，第 7~0 位为 0x22，以此类推；

4.4.2 测试验证

CPU GPIO 模拟，PCM X8 配置

./Linux_PH1A400_CFG.elf 4 PH4Led_Compress.bin

CPU GPIO 模拟，PCM X16 配置

./Linux_PH1A400_CFG.elf 5 PH4Led_Compress.bin

CPU GPIO 模拟，PCM X32 配置

./Linux_PH1A400_CFG.elf 6 PH4Led_Compress.bin

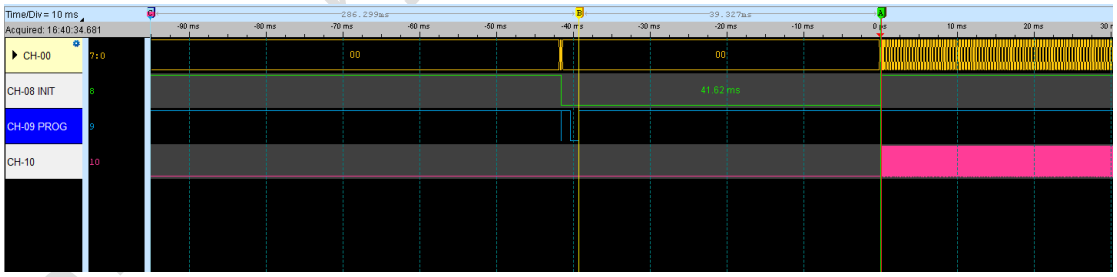

图 4-14 PH1A400 系列 FPGA 从动并行配置波形

4.5 从动并行配置回读

4.5.1 主动串行配置

从动并行配置读时序如图 4-15 所示，用户可以拉高 RDWRN 来回读配置数据，当 CSN 为低，RDWRN 为高时，BUSY 信号拉低表示回读 DATA[31:0]数据有效。

Preliminary Technical Report

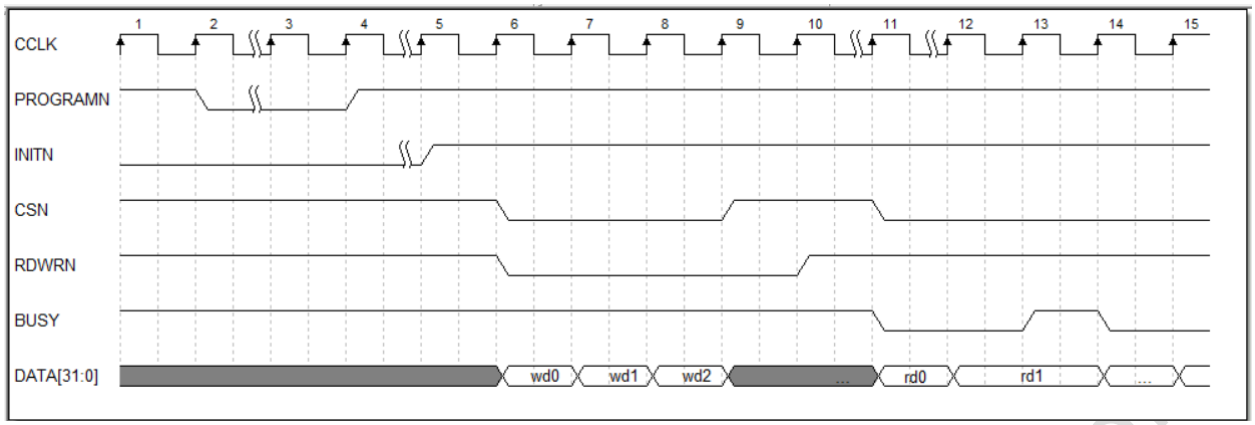

图 4-15 并行回读时序

对于回读，通过 vec 文件进行配置，先写再读。其中 vector 文件格式如下：

模式	从左到右
PCMx8	cclk csn rdwrn busy_n d7 d6 … … … d1 d0
PCMx16	cclk csn rdwrn busy_n d15 d14 … … … d1 d0
PCMx32	cclk csn rdwrn busy_n d31 d30 … … … d1 d0

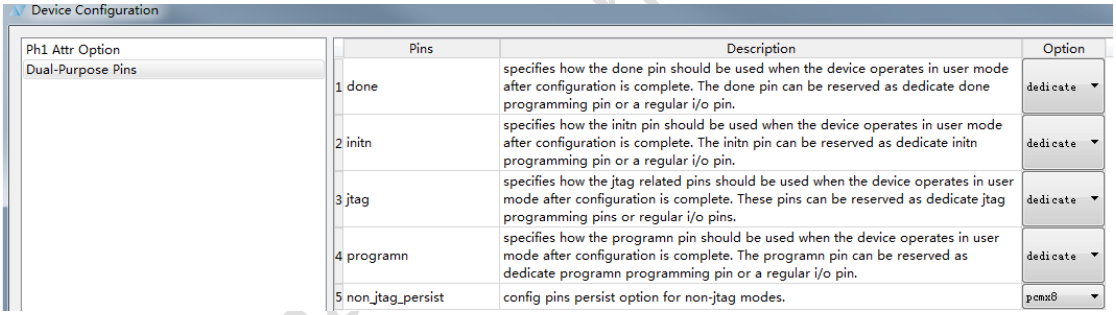

图 4-16 pcmx8 non_jtag_persist 设置

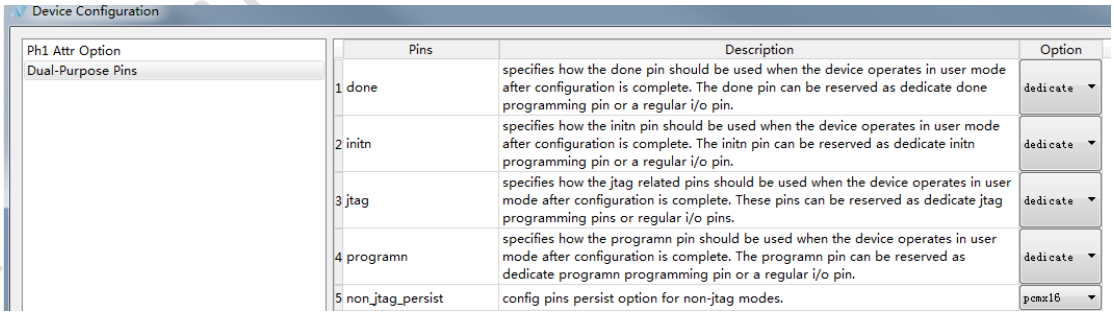

图 4-17 pcmx16 non_jtag_persist 设置

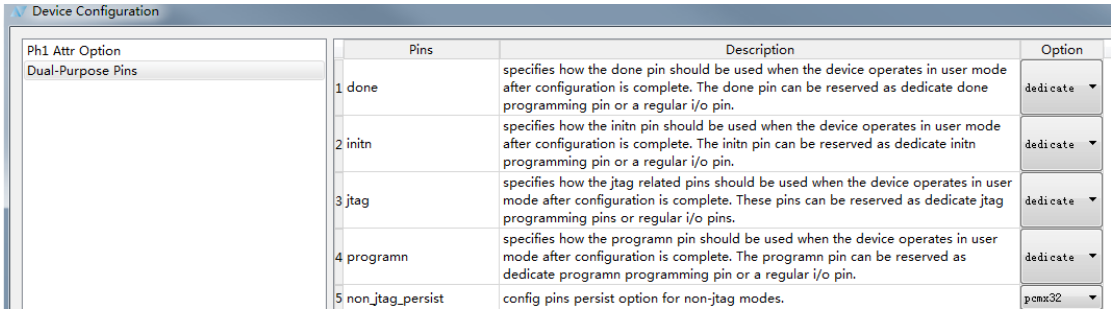

图 4-18 pcmx32 non_jtag_persist 设置

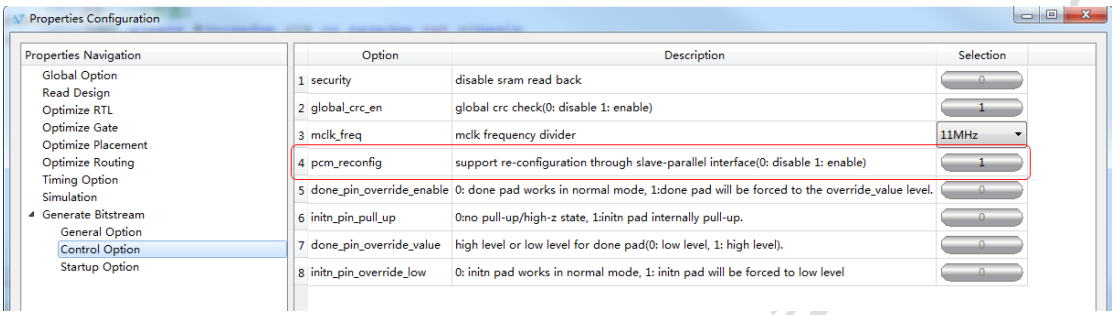

图 4-19 pcm_reconfig 使能

其中，配置完成后，处于 user 模式，即 User IO，因此在回读是需要将其弄成 dedicate IO，需要将 persist jtag 进行设置，如图 4-16、4-17、4-18 所示。

Reconfig PCM，配置如图 4-19 所示，将回读作为切换（需要访问权限）。

4.5.2 测试验证

Bin 文件生成，采用压缩模式，Bitstream 转 vector：

```
bitgen -bit bit0525.bit -compress -full_0
bit_to_vec -bit bit0525.bit -m pcmx8 -vec bit0525_pcmx8.vec
bit_to_vec -bit bit0525.bit -m pcmx8 -vec bit0525_pcmx8_ver.vec -verify

bit_to_vec -bit bit0525.bit -m pcmx16 -vec bit0525_pcmx16.vec
bit_to_vec -bit bit0525.bit -m pcmx16 -vec bit0525_pcmx16_ver.vec -verify

bit_to_vec -bit bit0525.bit -m pcmx32 -vec bit0525_pcmx32.vec
bit_to_vec -bit bit0525.bit -m pcmx32 -vec bit0525_pcmx32_ver.vec -verify
```

在回读 vector 文件中，对于 xxxxx，软件会自动忽略，对于 L 或者 H，在配置之前，先将其替换成 0 或者 1。

测试命令：

PCM X8 ReadBack

Preliminary Technical Report

```
./Linux_PH1A400_CFG.elf 7 bit0525_pcmx8.vec        bit0525_pcmx8_ver.vec
```

PCM X16 ReadBack

```
./Linux_PH1A400_CFG.elf 8 bit0525_pcmx16.vec       bit0525_pcmx16_ver.vec
```

PCM X32 ReadBack

```
./Linux_PH1A400_CFG.elf 9 PH4LED_spx32_prog.vec   PH4LED_spx32_verify.vec
```

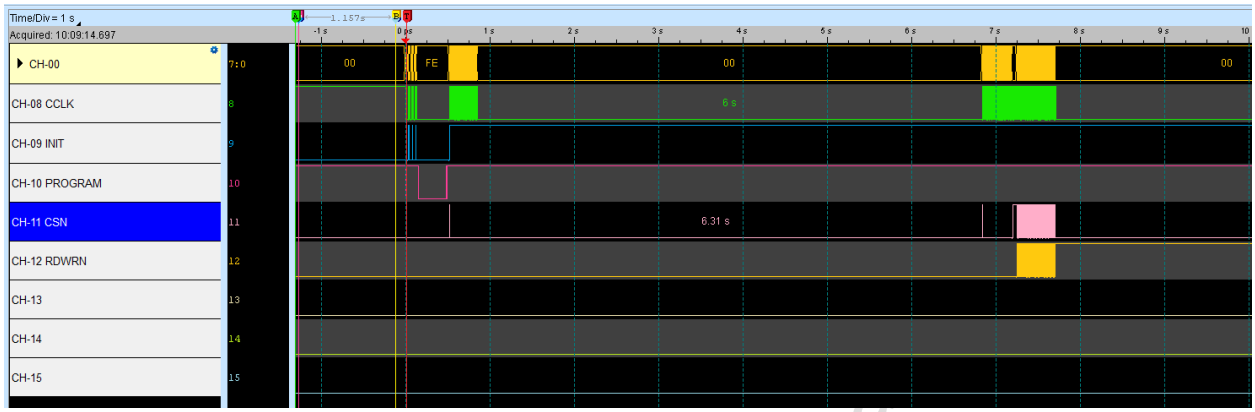

图 4-20 PCM 回读波形一

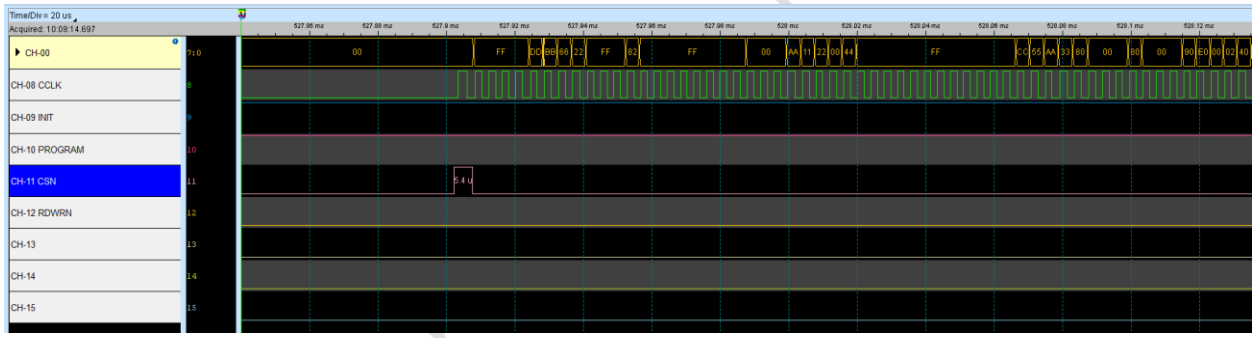

图 4-21 PCM 回读波形二

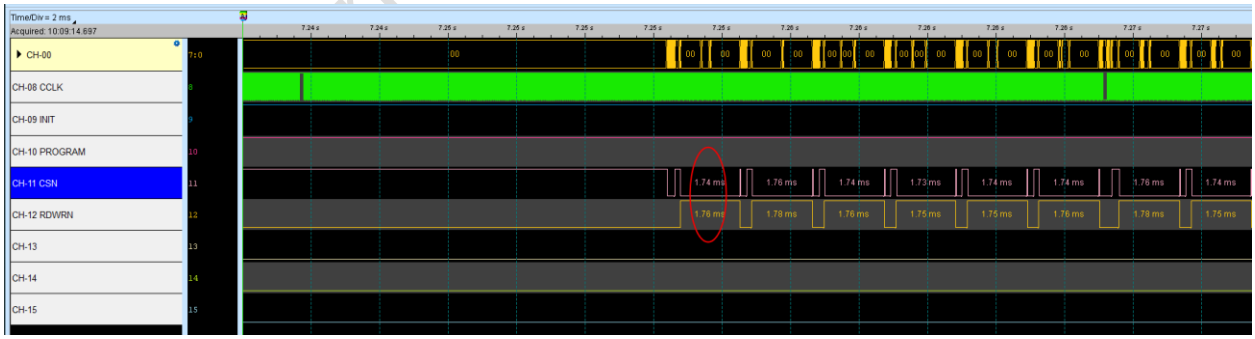

图 4-22 PCM 回读波形三

4.6 主动串行配置及其测试

4.6.1 主动串行配置

Mater Serial 配置模式如图 4-23 所示，CCLK 由外部上位机（如 CPU、MCU）提供。

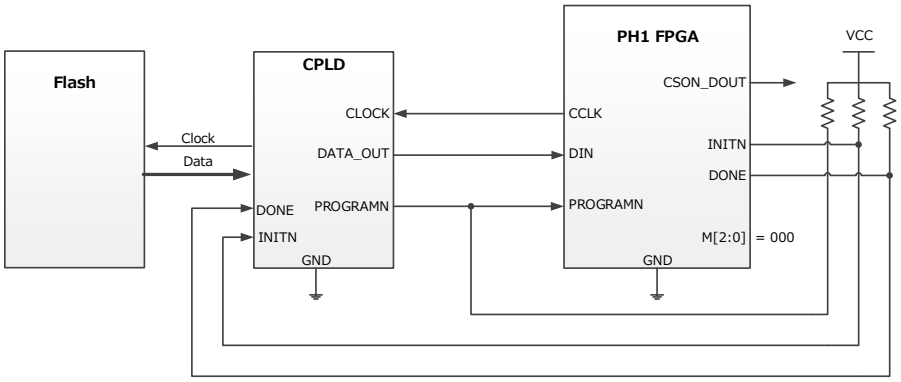

图 4-23 PH1A400 系列 FPGA 主动串行配置模式

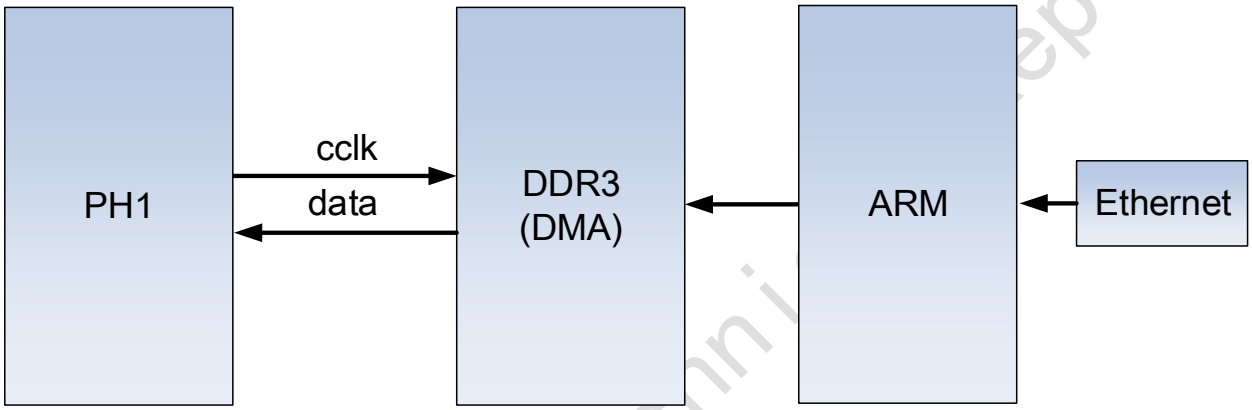

图 4-24 PH1 系列 FPGA MS/MP 模式测试平台

配置实现如图 4-24 所示，在配置的过程中，由于 PH1A400 为 Master，提供数据的读取所用的时钟，设计时将系统的 DDR 内存分成 2 块区域，一块为前 512MB 的空间（Linux 使用，在 uboot 和 dts 对其进行限制）、一块为后 512MB 的空间（用来存储配置数据）。

实现步骤如下：

- 系统启动；
- 打开 uio 设备；
- 读取 bin 文件数据，并根据模式将其进行相应转换，x1/x8/x6；
- 通过 fd = open ("/dev/mem", O_RDWR | O_SYNC)方式，访问物理内存（0x20000000）；
- 将转换后的数据，写入到物理内存；
- 对 ProgramN 进行复位处理；
- 读取 INITN 状态，若为 0 继续等待，若为 1 进行下一步操作；
- 发送 DMA 启动标志；
- PL 侧检测到 DMA 启动标志后，通过 AXI-HP 读取 PS 侧的内存（0x20000000）；
- 等待若干秒，关闭 uio；
- 配置完成；

Preliminary Technical Report

4.6.2 测试验证

Bin 文件生成，采用压缩模式

bitgen -bit xxx2.bit -bin PH4Led_Compress22M_Full.bin -g {osc_freq_div:0000010 } -compress -full_0

测试命令如下：

./Linux_PH1A400_CFG.elf 12 PH4Led_Compress22M_Full.bin

4.7 主动并行配置

4.7.1 主动串行配置

Master PCM 配置模式如图 4-25 所示，CCLK 由外部上位机（如 CPU、MCU）提供。

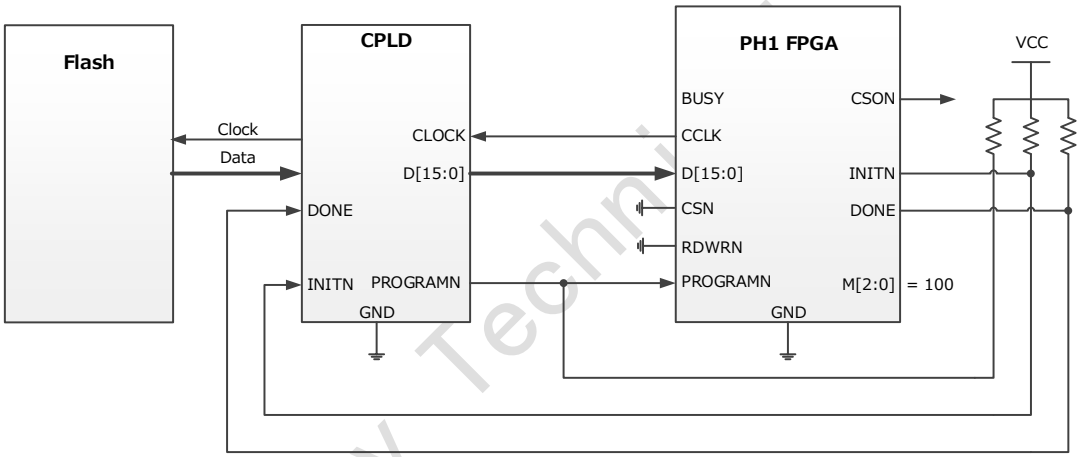

图 4-25 PH1A400 系列 FPGA Master PCM 模式

其中 M[2:0] = 100，实现方式与主动串行类似。

4.7.2 测试验证

22M bin 文件压缩模式

bitgen -bit xxx2.bit -bin PH4Led_Compress22M_Full.bin -g {osc_freq_div:0000010 } -compress -full_0

全 0 压缩版其中 X8 配置没问题，X16 配置 CRC ERROR，

对于 X16 暂时用

bitgen -bit xxx2.bit -bin PH4Led_Compress22M_Full.bin -g {osc_freq_div:0000010 } -compress

测试命令如下：

./Linux_PH1A400_CFG.elf 10 PH4Led_Compress22M_Full.bin

Preliminary Technical Report

```
./Linux_PH1A400_CFG.elf 11 PH4Led_Compress22M.bin
```

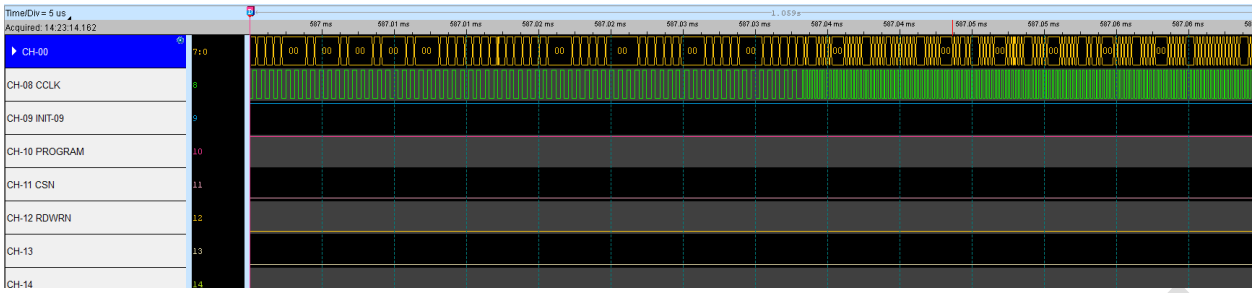

图 4-26 主动并行波形

4.8 从动快速配置及其测试

4.8.1 从动配置

从动并行、从动串行配置，支持最高速度 100MHz。前述章节已经展开，但配置速度因通过 CPU GPIO 模拟，速度较慢。为验证更高频率，对设计进行如下更新，如图 4-27 所示。

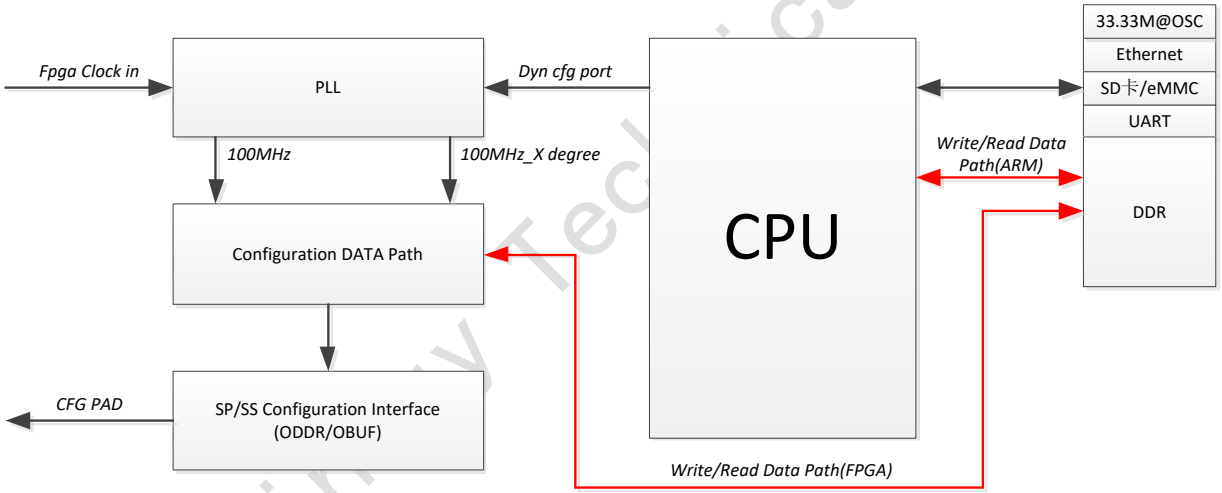

图 4-27 PH1A400 系列 FPGA 从动配置快速加载测试

实现步骤如下：

- 系统启动；
- 打开 uio 设备；
- 读取 bin 文件数据，并根据模式将其进行相应转换，x1/x8/x6；
- 通过 fd = open ("/dev/mem", O_RDWR | O_SYNC)方式，访问物理内存（0x20000000）；
- 将转换后的数据，写入到物理内存；
- 对 ProgramN 进行复位处理；
- 读取 INITN 状态，若为 0 继续等待，若为 1 进行下一步操作；
- 发送 DMA 启动标志；

Preliminary Technical Report

- PL 侧检测到 DMA 启动标志后，通过 AXI-HP 读取 PS 侧的内存（0x20000000），时钟由 FPGA PLL 提供，100MHz；
- 等待若干秒，关闭 uio；
- 配置完成；

4.8.2 测试验证

其中从动串行、从动并行 X8、X16、X32 命令行如下所示：

```
./Linux_PH1A400_CFG.elf 13 PH4Led_Compress.bin
./Linux_PH1A400_CFG.elf 14 PH4Led_Compress.bin
./Linux_PH1A400_CFG.elf 15 PH4Led_Compress.bin
./Linux_PH1A400_CFG.elf 16 PH4Led_Compress.bin
```

4.9 JTAG AJE/TDE 测试

有关 AJE/TDE 移植验证测试，可参考 AJE/TDE 相关应用文档。

Preliminary Technical Report

5 软件源码文件说明

打开压缩文件后，包含下述源码，支持 PH1 所有配置。

名	类型	大小
aje2vec.c	C 文件	17 KB
aje2vec.h	H 文件	1 KB
ajeutil.c	C 文件	6 KB
ajeutil.h	H 文件	2 KB
decode.h	H 文件	1 KB
decodel.c	C 文件	4 KB
fast_slave_pcm.c	C 文件	12 KB
fast_slave_pcm.h	H 文件	1 KB
fast_ss_cfg.c	C 文件	7 KB
fast_ss_cfg.h	H 文件	1 KB
gpio.c	C 文件	20 KB
gpio.h	H 文件	3 KB
hardware.c	C 文件	12 KB
ispvm_ui.c	C 文件	21 KB
ivm_core.c	C 文件	85 KB
lzw_lib.c	C 文件	16 KB
lzw_lib.h	H 文件	1 KB
main.c	C 文件	18 KB
main.c.bak	BAK 文件	15 KB
mp_pcm.c	C 文件	9 KB
mp_pcm.h	H 文件	1 KB
ms_cfg.c	C 文件	7 KB
ms_cfg.h	H 文件	1 KB
opcode.h	H 文件	3 KB
README.txt	TXT 文件	1 KB
sp_pcm.c	C 文件	9 KB
sp_pcm.h	H 文件	1 KB
sp_vec.c	C 文件	16 KB
sp_vec.h	H 文件	1 KB
ss_cfg.c	C 文件	7 KB
ss_cfg.h	H 文件	1 KB
vec.c	C 文件	15 KB
vec.h	H 文件	2 KB
vmopcode.h	H 文件	9 KB

图 34 PH1 系列配置软件代码

各模配置函数入口罗列如下：

配置模式	文件名	相关函数	描述
	main.c	主函数入口	通过命令行，进行切换
主动并行	mp_pcm.c	void MP_PCMX16(char *write_file_name);	主动并行 16 位配置函数
		void MP_PCMX8(char *write_file_name);	主动并行 8 位配置函数
主动串行	ms_cfg.c	void MS_SX1(char *write_file_name);	主动串行配置函数
从动并行	sp_pcm.c	void Slave_PCMX8_Cfg (char *write_file_name);	从动并行 8 位配置函数

Preliminary Report

		void Slave_PCMX16_Cfg (char *write_file_name);	从动并行 16 位配置函数
		void Slave_PCMX32_Cfg (char *write_file_name);	从动并行 32 位配置函数
从动串行	ss_cfg.c	void SlaveSerialX1_Cfg (char *write_file_name);	从动串行配置函数
从动串行(100MHz)	fast_ss_cfg.c	void Fast_Slave_Serial_X1_Cfg(char *write_file_name);	从动串行配置函数(100MHz)
从动并行回读	sp_vec.c	void Vector_PCMX8_ReadBack(char *write_file_name, char *read_file_name);	从动并行 8 位配置回读函数
		void Vector_PCMX16_ReadBack(char *write_file_name, char *read_file_name);	从动并行 16 位配置回读函数
		void Vector_PCMX32_ReadBack(char *write_file_name, char *read_file_name);	从动并行 32 位配置回读函数
从动并行回读	fast_slave_pcm.c	void Fast_Slave_PCMX8_Cfg(char *write_file_name);	从动并行 8 位配置回读函数(100MHz)
		void Fast_Slave_PCMX16_Cfg(char *write_file_name);	从动并行 16 位配置回读函数(100MHz)
		void Fast_Slave_PCMX32_Cfg(char *write_file_name);	从动并行 32 位配置回读函数(100MHz)

Preliminary Technical Report

版本信息

日期	版本	修订记录
2021/06/20	1.0	版本初始化

Preliminary Technical Report

版权所有©2021 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其它方式授予任 何知识产权许可。除安路科技在其产品的销售条款和条件中声明的责任之外，安路科技概不承担任何法律或非法律责任。安路科技对安路科技产品的销售和/或使用不作任何明示或暗示的担保，包括对产品的特定用途适用性、适销性或对任何专利权、版权或其它知识产权的侵权责任等，均不作担保。安路科技对文档中包含的文字、图片及其它内容的准确性和完整性不承担任何法律或非法律责任，安路科技保留修改文档中任何内容的权利，恕不另行通知。安路科技不承诺对这些文档进行适时的更新。